

Sampling Rate for Sinhala / Tamil Speaker Verification — Scientific Guide

Subject: Choice of sampling rate (F_s) for SL_SPV training; impact of mixing F_s across utterances.

Repository: voxceleb_trainer/

Author of analysis: Claude **Date:** 2026-05-14

Driving context: Sinhala corpus recorded at **44.1 kHz**; VoxCeleb at **16 kHz**; potential telephony domain at **8 kHz**.

The repo’s data loader hard-codes 16 kHz-derived constants and never resamples.

TL;DR — Read This First

- 1. Use 16 kHz, mono, PCM (int16 or float32) for every utterance** before it enters the training pipeline. It is the universal speaker-verification (SV) standard, covers all speaker-discriminative spectral information, and matches every pretrained model you might reuse (WavLM, ECAPA-TDNN, XLS-R, the existing MLP-Mixer V2 in this repo).
- 2. Downsample your 44.1 kHz Sinhala data to 16 kHz** with a proper anti-alias filter at corpus-build time. Do **not** rely on the loader to do it on the fly.
- 3. Do not mix sampling rates inside a single training run.** Mixed- F_s training causes the model to learn the recording-channel signature (which becomes a shortcut speaker cue), produces wrong-length analysis frames, and inflates training EER while collapsing real-world EER.
- 4. If you need to support 8 kHz telephony at *deployment* time, still train at 16 kHz,** but add **bandwidth-limited augmentation** (low-pass $\rightarrow$ decimate $\rightarrow$ upsample $\rightarrow$ optional codec) so the model sees both wide-band and narrow-band versions of the same speaker.
- 5. The data loader at `DatasetLoader.py:29` is silently 16 kHz-only.** Fix it (see Section 7) regardless of what F_s you choose.

1. What Sampling Rate Actually Controls (the Physics)

1.1 Nyquist–Shannon: the Only Hard Constraint

For a digital signal sampled at F_s , the highest frequency that can be represented without aliasing is $F_s/2$ (the Nyquist frequency). Anything above that present in the analog input will fold back into the passband as aliasing distortion unless removed by an anti-alias filter before sampling.

F_s (kHz)	Nyquist (kHz)	What you can represent
8.0	4.0	Narrow-band telephony. Loses energy above 4 kHz (sibilants /s/, // partly cut).
16.0	8.0	“Wide-band”. Covers full speech bandwidth used in SV literature.
22.05	11.025	Half of CD. Rarely used in SV.
44.1	22.05	CD-quality. $\approx 2.75\times$ the information SV needs; mostly music-sparkle content.
48.0	24.0	Pro-audio / video. Same conclusion as 44.1.

1.2 Where Speaker Identity Actually Lives Spectrally

Decades of speech-science work (Stevens 1998, Hansen & Hasan 2015, the NIST SRE programme, and every VoxCeleb-derived study) place speaker-discriminative information predominantly in:

Cue	Approx. band (Hz)	Why it matters
Fundamental frequency F_0	50–500	Vocal-fold mass/length $\rightarrow$ gender, age, pitch
Formants F_1 – F_3	200–3500	Vocal-tract length & shape, most individuating
Formants F_4 , F_5	3500–5000	Speaker-individual fine structure
Spectral tilt / glottal source	0–4000	Phonation type, breathiness
Higher harmonics, fricative noise	4000–8000	Some individual info in /s/, //, aspiration
Ultra-high band	>8000	Almost no speaker-discriminative information

Take-away: $F_s = 16$ kHz captures every band that carries non-trivial speaker information. Going to 44.1 kHz adds 6.05 kHz of bandwidth with essentially no incremental speaker-identity signal — but quadruples storage, IO, and FFT cost.

1.3 Why SV Benchmarks Settled on 16 kHz

- VoxCeleb1/2, CN-Celeb, VoxLingua, NIST SRE wide-band partitions, FLEURS, Common Voice (after standardisation): **all 16 kHz**.
- Mel-filterbanks (`n_mels = 40/80/128`) cover 0 – $F_s/2$ with logarithmic spacing; at 16 kHz the upper bin sits at 8 kHz, exactly where speaker info tapers off.
- Pretrained SSL encoders (WavLM, HuBERT, wav2vec-2, XLS-R, mHuBERT) are pretrained at 16 kHz; feeding them anything else silently mis-aligns positional encoding.
- 16 kHz is a “telco-superset”: you can simulate narrow-band by low-passing, but cannot recover wide-band after the fact.

1.4 What About 8 kHz?

Useful only when your *deployment* channel is genuinely 8 kHz (PSTN, AMR-NB cellular, some VoIP). Even then, the recommended practice is to **train at 16 kHz with bandwidth augmentation** and serve a single model — this consistently beats training a separate 8 kHz model on every published benchmark since ~ 2015 (cf. Snyder et al. NIST SRE19 system descriptions).

2. The Specific Situation: 44.1 kHz Sinhala Source

Your Sinhala corpus is recorded at 44.1 kHz. Three options exist:

Option	What happens	Verdict
A. Keep 44.1 kHz everywhere	Quadruples FFT cost; can't reuse pre-trained 16 kHz models; mel-filterbanks waste half their bins on a silent band above 8 kHz.	✗ Don't do this.
B. Downsample to 16 kHz once, at corpus-build time	Reversible loss only of the 8–22 kHz band (≈ 0 speaker info); aligns with all benchmarks and pretrained models.	Recommended.
C. Resample on-the-fly inside the data loader	Correct mathematically, but adds CPU cost per epoch, risks silent bugs, and makes augmentation ambiguous about target F_s .	△ Acceptable short-term.

2.1 The Downsampling Itself — Do It Right

Downsampling 44.1 $\rightarrow$ 16 kHz is **not exact** because $44100/16000 = 2.75625$ is not an integer. Two correct approaches:

Way 1 — sox (fastest, best filter, recommended for batch jobs):

```
sox input_44k.wav -r 16000 -c 1 -b 16 output_16k.wav rate -v -L
# -v = very high quality, -L = linear phase filter
```

Way 2 — librosa / torchaudio (in-Python, for pipelines):

```
import torchaudio
import torchaudio.functional as AF
wav, sr = torchaudio.load("input_44k.wav")           # (C, T) at 44100
wav = wav.mean(dim=0, keepdim=True)                 # force mono
wav16 = AF.resample(
    wav, orig_freq=sr, new_freq=16000,
    resampling_method="sinc_interp_kaiser",
    lowpass_filter_width=64, rolloff=0.99,
    beta=14.769656459379492,                          # Kaiser beta for 80 dB
    stop-band
)
torchaudio.save("output_16k.wav", wav16, 16000,
                encoding="PCM_S", bits_per_sample=16)
```

Common mistakes to avoid:

- Naive decimation (`wav[:, :3]`) without anti-alias filtering: aliases the 5.3–22 kHz band into 0–5.3 kHz as audible and inaudible spectral garbage.
- Resampling *after* loud normalisation: any rounding error rides on a maxed-out signal.
- Mixing `librosa.resample(..., res_type="kaiser_fast")` with `sox` across the dataset — see Section 3 for why this is dangerous.

2.2 Storage Choice

- **int16 PCM WAV** at 16 kHz: 32 kB/s. Lossless within int16 dynamic range. Use this unless you have a specific reason not to.
- **FLAC** at 16 kHz, level 5: $\approx$ half the size, lossless, natively supported by `soundfile/torchaudio`.
- Avoid Opus, MP3, AAC for *training-set storage*. Each transcode adds codec artefacts that the model can learn as spurious speaker features.

3. What Happens if You Train with Mixed Sampling Rates

3.1 The Mechanical Problem: Wrong Analysis Window

loadWAV in DatasetLoader.py:29 computes:

```
max_audio = max_frames * 160 + 240
```

Those constants assume 16 kHz with a 10 ms hop (160 samples) and 25 ms window. Feed it 44.1 kHz audio and:

- `max_frames = 200` no longer means “2 seconds”; it means **0.726 s** of audio at 44.1 kHz.
- Frame count, mel-bin frequencies, ASP pooling temporal dimension — **all become F_s -dependent** but the rest of the model assumes 16 kHz.

The script will **not crash**. It silently trains on mis-framed inputs — EER never reaches its potential.

3.2 The Information-Content Problem

- Mixing 8 kHz and 16 kHz audio in the same batch: the 8 kHz samples have **zero energy** above 4 kHz (or aliased garbage). The model sees upper mel bins as “near-silent” only for *some* utterances, correlating silence with the recording channel rather than the speaker.
- Mixing 16 kHz and badly-downsampled 44.1 kHz: filter ripple and roll-off in the 6–8 kHz band become a non-speaker shortcut cue.

3.3 The Shortcut-Learning Problem (the Big One)

Neural networks are **shortcut learners** (Geirhos et al., *Nat. Mach. Intell.* 2020). Given any feature that correlates with the label but is not the intended concept, the network will exploit it.

Suppose 80% of speaker A’s utterances are 44.1 kHz (studio) and 80% of speaker B’s are 8 kHz (phone). The AAM-Softmax classifier learns “if the audio has zero energy above 4 kHz, this is probably speaker B.” Validation EER on same-condition trials looks great. The moment you evaluate on B’s studio voice, EER collapses.

This is *exactly* the mechanism that has caused multiple published SV systems to score well on VoxCeleb1-O and fail on telephony deployment.

3.4 The Pretrained-Model Misalignment Problem

Initialising from the 16 kHz MLP-Mixer V2 checkpoint then feeding it 44.1 kHz audio shifts the SincConv learnable band centres by $44.1/16 = 2.75625\times$. Every downstream layer sees the wrong receptive field. The checkpoint is *worse than a random init* in this case.

3.5 The Augmentation-Mix Problem

AugmentWAV convolves audio with RIR files and mixes in MUSAN noise at 16 kHz. If audio is 44.1 kHz, the convolution is dimensionally legal but **acoustically nonsense**: the RIR’s 1-second impulse becomes a 0.36-second impulse, simulating a tiny room; the noise spectrum’s 4 kHz edge sits at 1.45 kHz of your input.

3.6 The Trial-Pair Scoring Problem

If enrol is 16 kHz and test is 44.1 kHz (or vice-versa), cosine similarity is biased by bandwidth mismatch even for the same speaker. Empirically this can add 2–5% absolute EER on a corpus with mixed sources unless every trial pair is forced to identical F_s *and* identical resampler chain.

4. Quantified Expectation of the Damage

Order-of-magnitude estimates drawn from the SV literature and the architecture choices in this repo (relative changes versus a clean 16 kHz baseline):

Scenario	Expected EER impact	Confidence
All training & test audio at 16 kHz	baseline	–
All audio resampled 44.1 → 16 kHz, high-quality filter	$\leq +0.1\%$ absolute	High
Mixed 16 kHz + 8 kHz, no bandwidth aug	+2 to +5% absolute on cross-bandwidth trials	High
Mixed 16 kHz + 44.1 kHz, both fed raw into loader	+3 to +10% absolute , possibly NaN	High
Trained 16 kHz + bandwidth aug, evaluated on 8 kHz telephony	+0.5 to +1.5% absolute	High
Trained 16 kHz, evaluated on 8 kHz with no aug	+3 to +8% absolute	High
MinDCF impact	$\approx 1.5\text{--}2\times$ the EER swing	Medium

5. Recommended Training Recipe for the SL Corpus

5.1 Corpus-Build Stage (One-Time)

- Standardise everything to **16 kHz, mono, 16-bit PCM WAV (or FLAC)**.
 - Sinhala 44.1 kHz → `sox ... rate -v -L 16000`.
 - Any 8 kHz telephony: keep as 8 kHz on disk; label `narrow_band=true` in metadata. Upsample to 16 kHz inside the loader; never archive the upsampled version as the primary file.
 - Tamil OpenSLR (mostly 16 kHz): already fine.
- Apply a soft VAD (e.g. `silero-vad` or `webrtcvad` aggressiveness 2) to trim leading/trailing silence > 300 ms. Do *not* aggressively remove internal silence — speaker rhythm carries information.
- Loudness-normalise to -23 LUFS (`pyloudnorm`) or peak-normalise to -1 dBFS. Pick one and stick with it.
- Write a `metadata.tsv` with columns:
 `utt_id`, `speaker_id`, `language`, `channel`, `source`, `duration_s`, `original_fs`.

5.2 Loader Stage (Every Batch)

Patch `DatasetLoader.py` so that `loadWAV`:

- accepts `sample_rate` (target F_s , default 16000) as a parameter,
- reads native F_s from `soundfile`,
- resamples via `torchaudio.functional.resample` or `scipy.signal.resample_poly` if mismatched,
- raises a loud warning the first time per process if any resampling occurs,
- recomputes `max_audio` from `int(max_frames × 0.01 × target_sr + 0.015 × target_sr)`.

Minimal patch sketch:

```
def loadWAV(filename, max_frames, evalmode=True, num_eval=10,
            target_sr: int = 16000):
    hop_s, win_s = 0.010, 0.025
    max_audio = int(max_frames * hop_s * target_sr + win_s * target_sr)

    audio, sr = soundfile.read(filename, dtype="float32", always_2d=False)
    if audio.ndim > 1:
        audio = audio.mean(axis=1)  # force mono
    if sr != target_sr:
        from math import gcd
        g = gcd(sr, target_sr)
        audio = signal.resample_poly(audio, target_sr // g, sr // g)

    audiosize = audio.shape[0]
    if audiosize <= max_audio:
        shortage = max_audio - audiosize + 1
        audio = numpy.pad(audio, (0, shortage), mode="constant")
        audiosize = audio.shape[0]
    # ... rest unchanged
```

Two improvements bundled in: (a) `mode="constant"` instead of `'wrap'` (avoids fabricating repeated content for short clips), and (b) explicit mono conversion.

5.3 Augmentation Stage — Simulate the Channels You Don't Have

```
def bandwidth_aug(audio, sr=16000, p=0.3):
    if random.random() > p:
        return audio
    cutoff = random.choice([3400, 3700, 4000])
    sos = signal.butter(8, cutoff, btype="low", fs=sr, output="sos")
    audio = signal.sosfilt(sos, audio)
    # Decimate to 8 kHz then back to 16 kHz
    audio = signal.resample_poly(audio, 1, 2)
    audio = signal.resample_poly(audio, 2, 1)
    # Optional: mu-law companding to simulate G.711
    if random.random() < 0.5:
        audio = numpy.sign(audio) * \
            numpy.log1p(255 * numpy.abs(audio)) / numpy.log1p(255)
    return audio.astype("float32")
```

With $p \approx 0.3$, a 16 kHz-trained model becomes bandwidth-robust without a separate 8 kHz training run.

5.4 Evaluation Stage

- Run separate eval lists for **wide-band**, **narrow-band**, **cross-bandwidth**, and **code-switched** subsets.
- The cross-bandwidth subset is the honest test of whether bandwidth augmentation worked.
- Report EER and MinDCF for each subset.

6. Decision Matrix

Your data	Your deployment	Recommended F_s	What to fix
All Sinhala 44.1 kHz; all Tamil 16 kHz	Mobile app, broadband	16 kHz (down-sample Sinhala)	§5.2 loader patch
Mix of studio and broadcast (44.1)	Same as above	16 kHz	§5.2 loader patch
Adds telephony 8 kHz	Mixed broadband + phone	16 kHz + bandwidth aug	§5.2 + §5.3
Pure call-centre 8 kHz only	Call-centre only	Stay at 8 kHz ; retrain from scratch	§5.2 patch + retrain mel configs
Cross-language code-switch eval	Anywhere	16 kHz ; same recipe	§5.2 + multi-lingual splits

7. Concrete Fixes to Land in This Repo

Minimum changes needed before any SL training run can be trusted:

#	File	Change	Why
1	DatasetLoader.py:26-55	Add <code>target_sr</code> param to <code>loadWAV</code> ; resample if mismatch; recompute <code>max_audio</code> from <code>target_sr</code>	Root bug. Everything else follows.
2	DatasetLoader.py:38	Replace <code>'wrap'</code> padding with <code>'constant'</code> + small noise	Wrap-pad fabricates speaker-specific repetition cues for short clips.
3	AugmentWAV.__init__	Read RIR/MUSAN at construction time and resample to <code>target_sr</code> if needed	Otherwise augmentation is at the wrong F_s .
4	All configs/*.yaml	Add <code>sample_rate: 16000</code> and pipe through to loader	Make the choice explicit, not hidden in a constant.
5	DatasetLoader.py	Add <code>bandwidth_aug</code> augmentation behind a config flag	Robustness to deployment-channel mismatch.
6	corpus_build.py (new)	One-shot script: walk source dir, transcode to 16 kHz mono 16-bit PCM via sox, write <code>metadata.tsv</code> and <code>train_list.txt</code>	Make corpus-prep reproducible.
7	check_corrupted_audio.py	Extend to also report native F_s distribution across the corpus	Surface mixed- F_s corpora at scan time, not at epoch 30.

8. Frequently Asked Questions

Q: My Sinhala recordings are 44.1 kHz, 24-bit, stereo, studio quality. Won't I lose information downsampling to 16 kHz mono 16-bit? Yes, but not information that matters for SV. You lose (a) the 8–22 kHz spectral band (≈ 0 speaker info), (b) the second channel (typically room-mic redundancy), (c) the lower 8 bits of dynamic range (below the noise floor of typical speech). Empirically, EER does not budge.

Q: Why not train at 44.1 kHz to “preserve as much as possible”? Three reasons: (i) you cannot use any pretrained model; (ii) FFT/mel-spec compute roughly triples; (iii) the upper bins carry mostly recording-environment cues that the model latches onto as shortcut features — actively *hurting* generalisation.

Q: My VAD removed too much; should I lower the threshold? Yes. Use a soft VAD that only trims edges. Internal pauses, breath, lip noise carry speaker information. Don't aggressively de-silence.

Q: Can I train one model that handles 8/16/44.1 kHz inputs? One model, yes — but the *one* input rate it sees must be 16 kHz. Resample everything to 16 kHz at the loader, then use bandwidth augmentation (§5) so the signal sometimes “behaves” like 8 kHz telephony. This is what every modern production SV system does.

Q: 24 kHz is a common middle ground (used by some TTS systems). Should I use it? No SV benchmark uses it; no pretrained SV model uses it; it offers no measurable SV gain

over 16 kHz. Use 16 kHz.

Q: Should I time-stretch utterances as augmentation? Speed perturbation ($\pm 10\%$, e.g. with `sox tempo 0.9` and `tempo 1.1`) is a *good* standard augmentation; it is conceptually independent of sampling rate. Apply it *after* resampling to 16 kHz.

9. References & Further Reading

Stevens, K. N. (1998). *Acoustic Phonetics*. MIT Press. — formant-frequency speaker individuality.

Snyder, D. et al. (2018). “X-vectors: Robust DNN Embeddings for Speaker Recognition.” *ICASSP*. — 16 kHz standard, wide-band/narrow-band considerations.

Chen, S. et al. (2022). “WavLM: Large-scale self-supervised pretraining for full stack speech processing.” *IEEE J. Sel. Topics Signal Process.* — 16 kHz SSL.

Geirhos, R. et al. (2020). “Shortcut learning in deep neural networks.” *Nat. Mach. Intell.* — why mixed- F_s training fails generalisation.

Smith, J. O. (2002). “Digital Audio Resampling Home Page” — engineering of high-quality resampling.

Conneau, A. et al. (2022). “FLEURS: Few-shot Learning Evaluation of Universal Representations of Speech.” — Sinhala/Tamil multilingual eval, all 16 kHz.

NIST SRE evaluation plans (2018, 2019, 2021) — bandwidth handling guidance for production SV.

Bottom line: Standardise the entire Sri Lankan corpus to **16 kHz mono PCM** at build time; patch the data loader to resample defensively and recompute frame constants from `target_sr`; add bandwidth augmentation if 8 kHz telephony is in scope; **never** mix native sample rates inside a training run. Following this avoids the silent-corruption mode that the current loader is wide open to, and aligns the project with every pretrained checkpoint and every public benchmark you will want to compare against.